PROYECTO INTEGRADOR · OPTATIVA II

ServiX

Gestión de servicios técnicos con roles, chat con inteligencia artificial y entrega continua

Juan Esteban Olaya Muñoz · Developer

Nicolas Quiroz Gomez · Product Owner

Juan Sebastian Garzon Suarez · Scrum Master

Universidad de San Buenaventura · Optativa II
Docente: Ing. Jairo Armando Salcedo Aranda

Equipo ServiX · Noviembre de 2026



Render con datos de demostración (Robolectric)

AGENDA

Seis bloques para recorrer ServiX de principio a fin



1	**Contexto y solución**	Diapositivas 1 a 6	**5 min**
2	**Recorrido guiado de la app**	Diapositivas 7 a 15	**8 min**
3	**Arquitectura, datos e IA**	Diapositivas 16 a 18	**4 min**
4	**Seguridad del código**	Diapositiva 19	**1 min**
5	**Proceso: Scrum, CI/CD y Git**	Diapositivas 20 a 23	**4 min**
6	**Calidad, balance y cierre**	Diapositivas 24 a 29	**5 min**

QUÉ SIGNIFICA

Si en algún momento se pierden, miren qué bloque estamos viendo: cada uno responde una pregunta distinta.

Sin un sistema, el técnico pierde el control de su trabajo

- **Un día cualquiera.** Imaginemos a un técnico de aires acondicionados: recibe pedidos por WhatsApp, anota direcciones en un cuaderno y cobra en efectivo. Al final de la semana no sabe con certeza cuántos trabajos quedaron pendientes ni cuánto dinero le deben.
- **El cliente también pierde.** Quien contrató el servicio llama para preguntar si el técnico ya va en camino, porque no tiene ninguna forma de ver el estado de su pedido.
- **El problema de fondo.** La información está dispersa, no hay historial, no hay roles y no existe un canal claro para quejas o dudas.



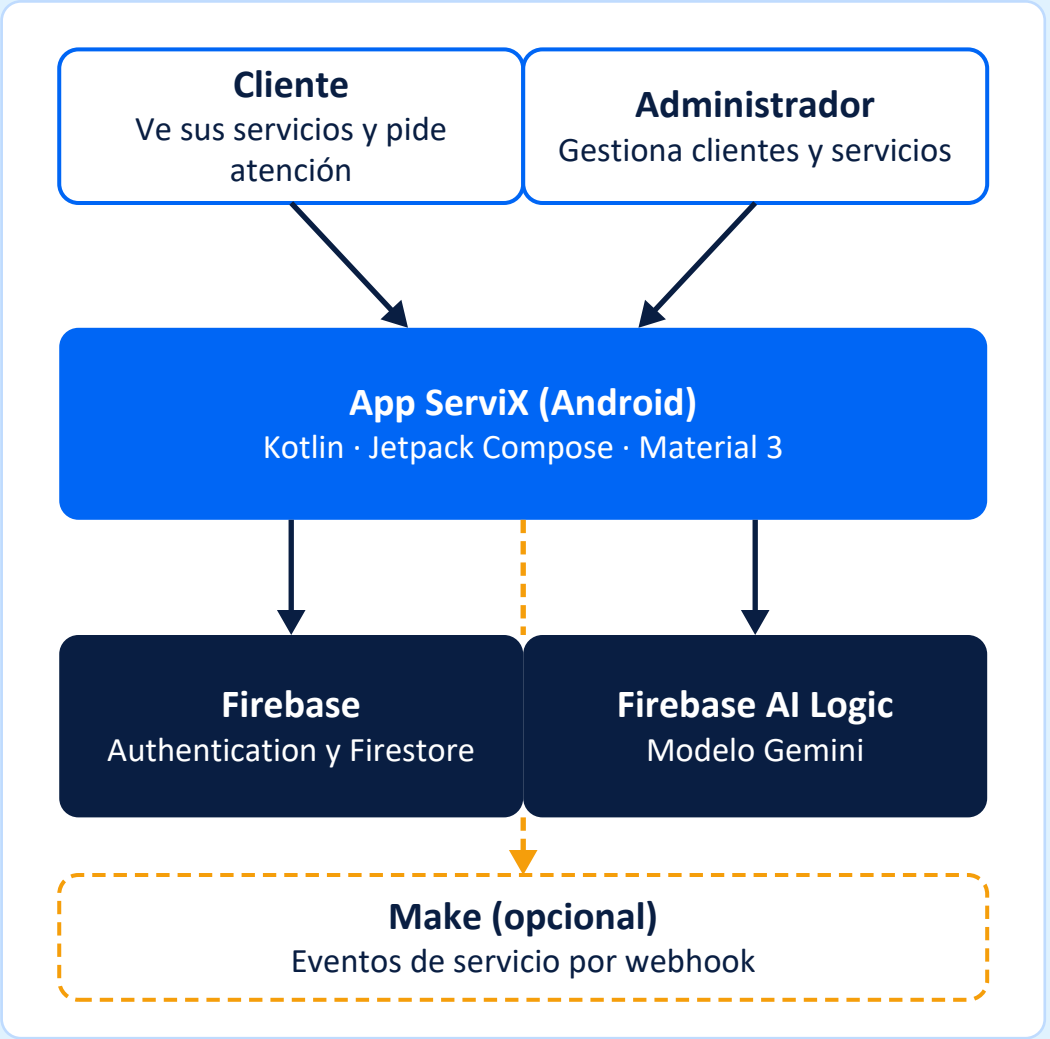
QUÉ SIGNIFICA

ServiX junta clientes, trabajos, cobros y atención en un solo lugar. El ejemplo del técnico es ilustrativo.

LA SOLUCIÓN

ServiX: una app Android para gestionar servicios técnicos

- **Qué es.** Una aplicación Android hecha con Kotlin y Jetpack Compose. El técnico, que es el administrador, registra clientes y órdenes de servicio y sigue cada trabajo por estados: pendiente, en proceso, completado o cancelado.
- **Qué la hace distinta.** Incluye un historial financiero, un centro de avisos y un asistente de chat con inteligencia artificial que atiende dudas y deriva los casos delicados a una persona.
- **Dónde vive la información.** Las cuentas están en Firebase Authentication y los datos en Cloud Firestore, protegidos con reglas de seguridad por rol.



QUÉ SIGNIFICA

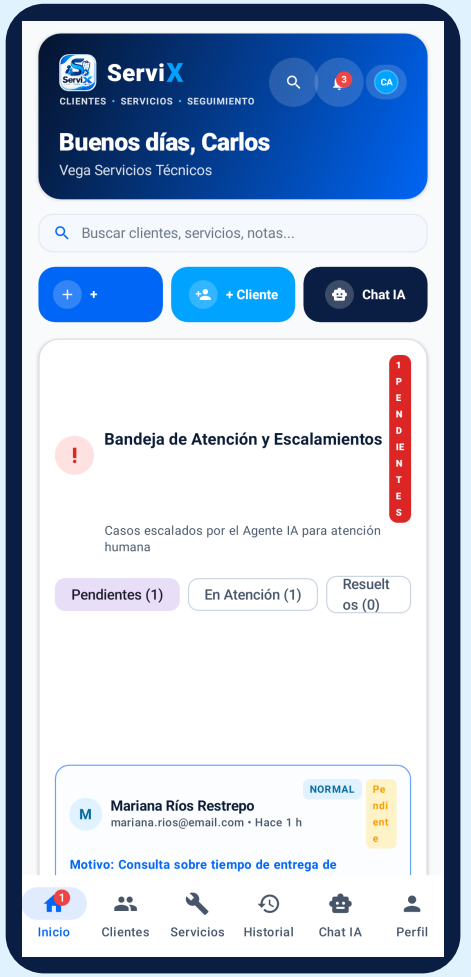
Es la libreta del técnico, pero ordenada, con control de acceso y con ayuda de IA.

Dos roles, dos experiencias distintas

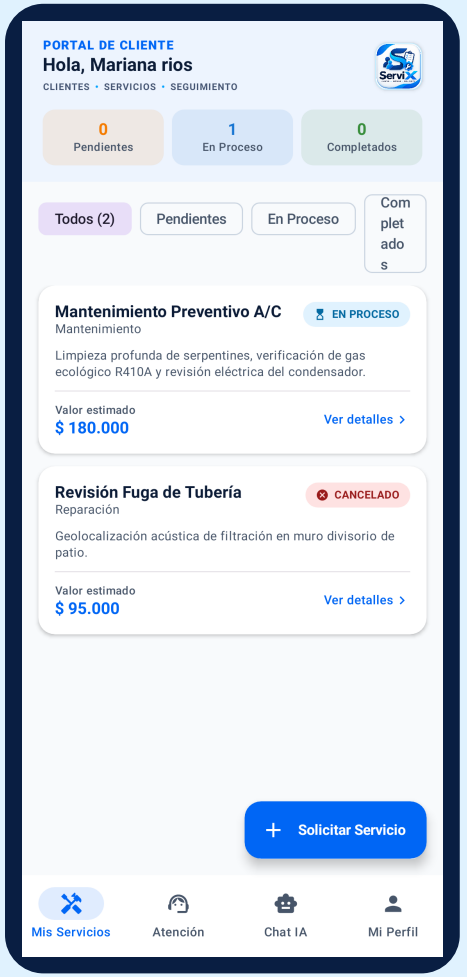
- **Administrador (el técnico).** Gestiona todos los clientes y servicios, cambia estados, consulta el historial financiero, atiende los casos escalados desde el chat y ajusta su perfil y sus metas.
- **Cliente.** Ve solo sus propios servicios, puede solicitar uno nuevo, conversa con el asistente de IA y pide atención humana cuando lo necesita. No ve datos de otros clientes.
- **Quién decide el rol.** El rol se guarda en la cuenta y las reglas de Firestore lo comprueban en el servidor; no basta con esconder botones en la pantalla.

QUÉ SIGNIFICA

Cada persona ve solo lo que le corresponde, y eso lo garantiza el servidor, no solo la interfaz.



Administrador



Cliente

Render con datos de demostración (Robolectric)

OBJETIVOS Y ALCANCE

Qué buscamos lograr y hasta dónde llegamos

Objetivo general. Construir una aplicación móvil funcional, segura y probada que ordene la gestión de clientes y servicios técnicos, aplicando Scrum, control de versiones e integración continua.

Incluye

- Autenticación con dos roles
- Clientes y servicios con estados
- Historial financiero y avisos
- Chat con IA y escalamiento a una persona
- Reglas de seguridad en Firestore
- Pipeline de integración continua

No incluye (por ahora)

- Pruebas ejecutadas en un dispositivo
- Chat verificado contra Gemini real
- Pagos en línea
- Versión iOS o web
- Backend propio para asignar roles

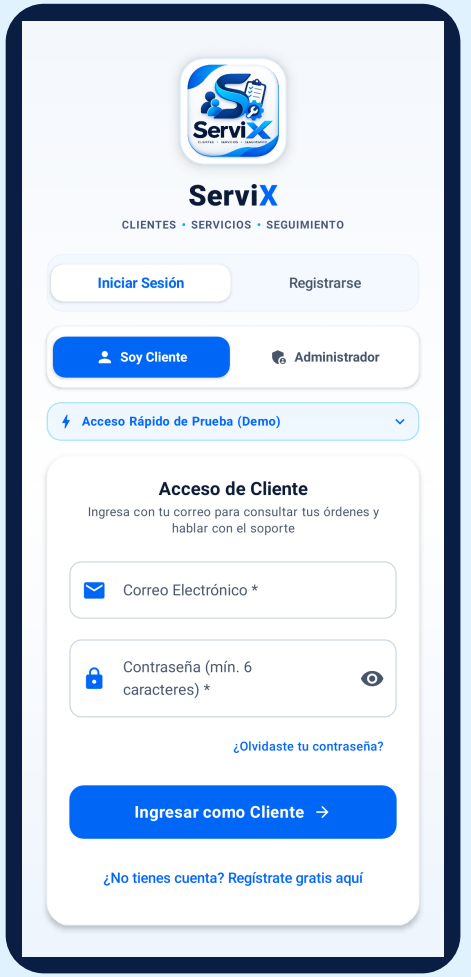
QUÉ SIGNIFICA

Decimos con claridad qué funciona hoy y qué queda como trabajo futuro: así se evalúa con honestidad.

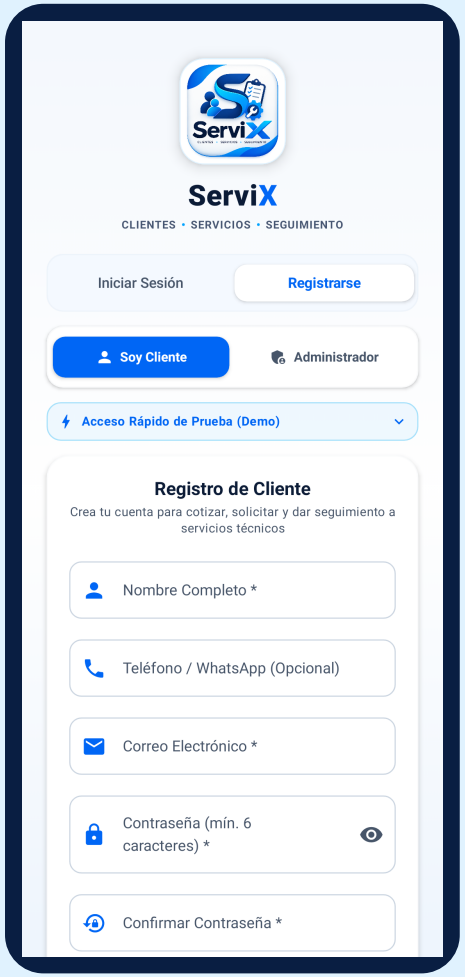
RECORRIDO · ACCESO

Entrar a ServiX: acceso y registro con correo

- **Qué ve el usuario.** El logo y dos pestañas: Iniciar sesión y Registrarse. Elige si entra como Cliente o como Administrador, y escribe su correo y una contraseña de al menos 6 caracteres.
- **Qué acción hace.** Pulsa Ingresar, o crea su cuenta con nombre, teléfono opcional, correo y contraseña confirmada. También puede recuperar su contraseña.
- **Qué valor aporta.** Firebase Authentication verifica la identidad, y el rol sale de la cuenta guardada: si no coincide con el elegido, se rechaza el acceso.



Iniciar sesión



Registro

Render con datos de demostración (Robolectric)

QUÉ SIGNIFICA

Nadie entra con un rol que no le pertenece. El atajo de prueba solo existe en compilaciones debug.

Inicio: el resumen del día y accesos directos

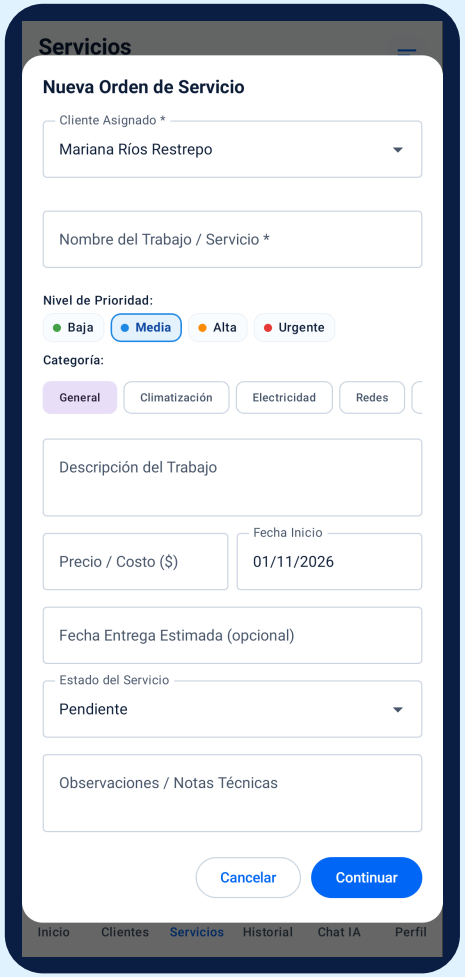
- **Qué ve.** Un saludo con el nombre del negocio, un buscador global de clientes y servicios, tres botones rápidos (nuevo servicio, nuevo cliente y chat IA) y la bandeja de atención.
- **Qué hace.** Desde el botón de nuevo servicio abre el formulario de la segunda captura: elige el cliente, escribe el trabajo, define prioridad y categoría, fechas y precio, y guarda.
- **Qué valor aporta.** Registrar un trabajo toma pocos pasos y queda ligado a un cliente, con prioridad y fecha, listo para su seguimiento.

QUÉ SIGNIFICA

Lo urgente está a un toque de distancia: el técnico empieza el día sabiendo qué tiene pendiente.



Inicio (administrador)

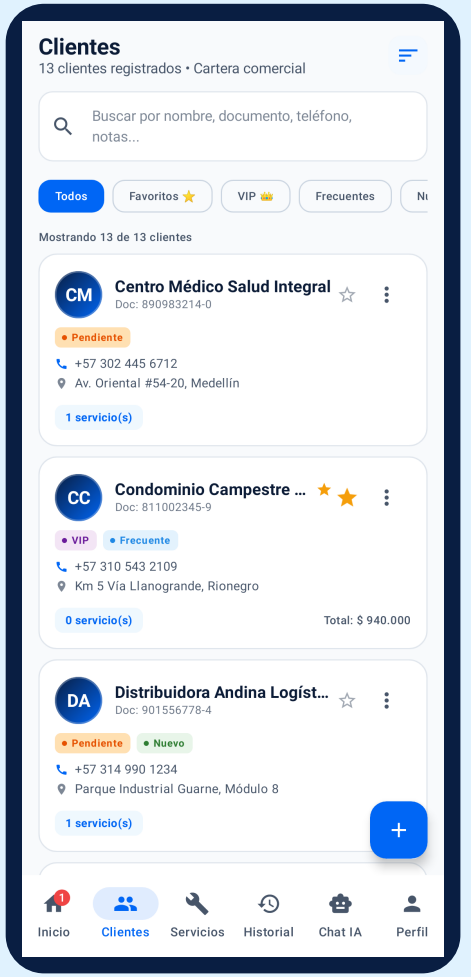


Nueva orden de servicio

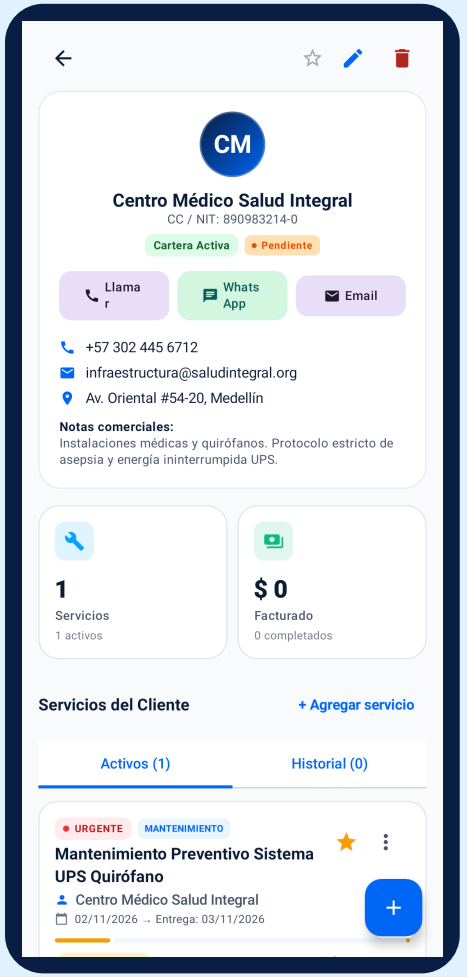
Render con datos de demostración (Robolectric)

Clientes: una cartera ordenada y a un toque de contacto

- **La lista.** Muestra 13 clientes de demostración con búsqueda por nombre, documento, teléfono o notas, y filtros: Todos, Favoritos, VIP y Frecuentes. Cada tarjeta indica etiquetas y cuántos servicios tiene.
- **El detalle.** Reúne los datos del cliente, sus notas comerciales y tres botones para llamar, escribir por WhatsApp o enviar un correo.
- **El valor.** Debajo aparecen sus servicios activos y su historial, y se puede agregar uno nuevo: el técnico conoce el contexto antes de ir.



Lista de clientes



Detalle de un cliente

Render con datos de demostración (Robolectric)

QUÉ SIGNIFICA

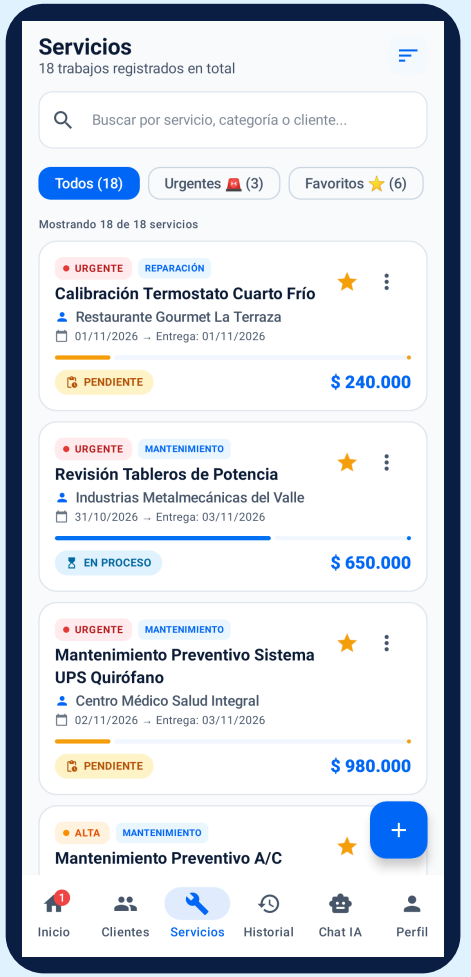
El cliente deja de ser un contacto suelto en el teléfono y pasa a tener una historia completa.

Servicios: cada trabajo con prioridad, estado y progreso

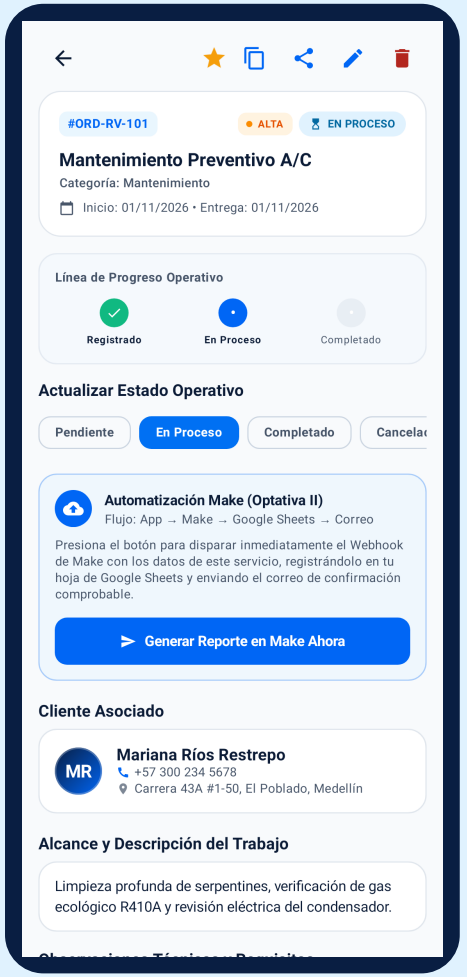
- **La lista.** 18 trabajos de demostración, con filtros por urgentes y favoritos. Cada tarjeta muestra prioridad, categoría, cliente, fechas, precio y una barra de avance.
- **El detalle.** Una línea de progreso: Registrado, En proceso, Completado. Con un toque se cambia el estado, incluso a Cancelado.
- **La automatización.** El detalle incluye un botón para enviar el evento a Make, una herramienta que puede generar reportes y correos. Está programado y probado con un doble de prueba, pero no se ejecutó contra Make real.

QUÉ SIGNIFICA

El estado de cada trabajo siempre está claro para quien lo atiende y, en el portal, para quien lo contrató.



Lista de servicios



Detalle de un servicio

Render con datos de demostración (Robolectric)

Historial: cuánto se cobró, cuánto falta y cuánto se perdió

- **Qué ve.** El Centro Financiero, con filtros de esta semana, este mes y tres meses, y cuatro indicadores: ingresos realizados, dinero pendiente, ticket promedio y cancelados no facturados.
- **La cartera.** Una barra con la distribución de los 18 servicios de demostración: 7 completados, 9 en curso y 2 cancelados, y debajo la trazabilidad de cada trabajo.
- **El valor.** Responde la pregunta que el cuaderno no podía responder: cuánto dinero me deben. Las cifras de la captura son ficticias.

QUÉ SIGNIFICA

Convierte los trabajos registrados en información para decidir, sin hacer cuentas a mano.



Centro financiero

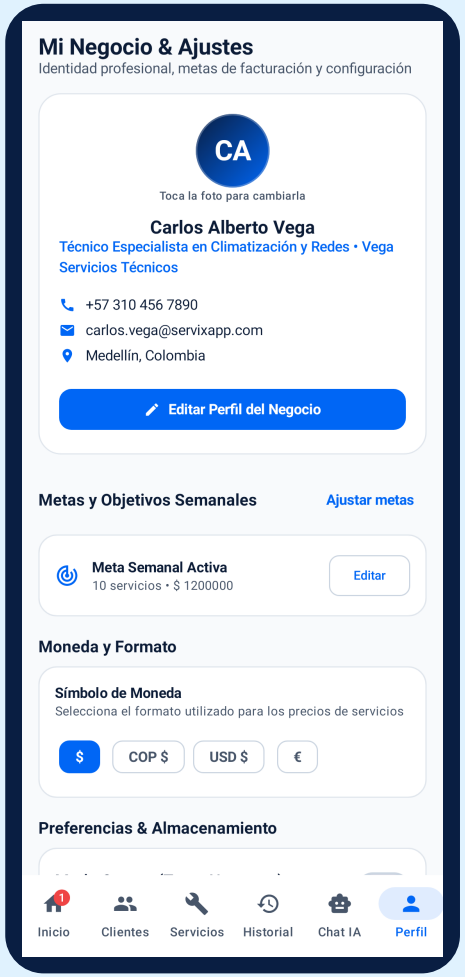
Render con datos de demostración (Robolectric)

Perfil: la identidad y las metas del negocio

- **Qué ve.** El nombre, la especialidad y la empresa del técnico, con sus datos de contacto y una foto que puede cambiar. Más abajo, las metas semanales de servicios e ingresos.
- **Qué hace.** Edita su perfil con un botón y escoge el formato de moneda: pesos colombianos, dólares u otra opción.
- **Qué valor aporta.** Personaliza la app para cada negocio y le da al técnico un objetivo medible, como el ejemplo de 10 servicios por semana.

QUÉ SIGNIFICA

La aplicación se adapta al negocio del técnico, no al revés.



Perfil y ajustes

Render con datos de demostración (Robolectric)

Chat con IA: respuestas al instante para ambos roles

- **Qué ve.** Un chat con el Asistente ServiX, un saludo y preguntas sugeridas, por ejemplo cómo van mis servicios o qué significa el estado En proceso.
- **Qué hace.** Escribe su consulta y el asistente responde con los datos de esa persona. Si el mensaje es una queja, una cancelación o pide un asesor, el sistema lo escala a una persona.
- **Qué valor aporta.** Resuelve dudas sin interrumpir al técnico y no inventa respuestas: si la IA falla, muestra un error claro. Gemini real no se ejecutó en dispositivo.

QUÉ SIGNIFICA

La IA atiende lo simple y una persona atiende lo importante.



Chat (administrador)

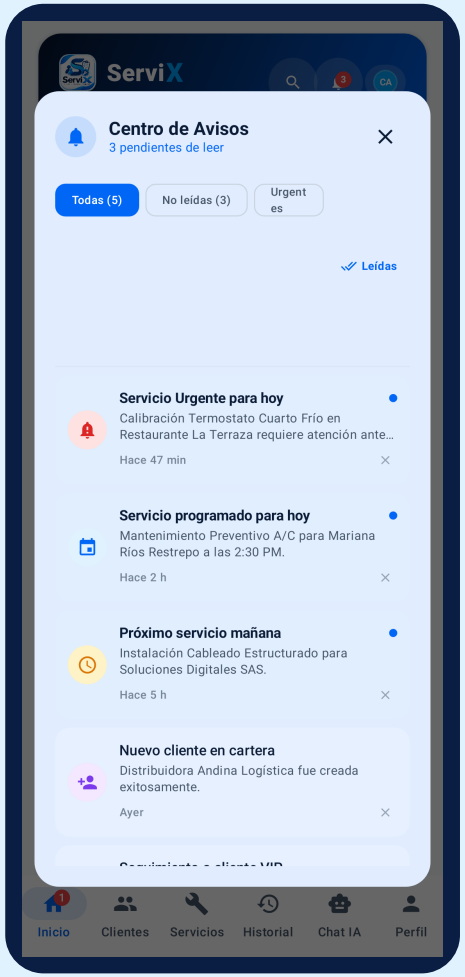


Chat (cliente)

Render con datos de demostración (Robolectric)

Avisos: nada urgente se queda sin leer

- **Qué ve.** El Centro de Avisos indica cuántos hay pendientes de leer, tres en el ejemplo, con filtros: Todas, No leídas y Urgentes.
- **Qué avisos hay.** Servicio urgente para hoy, servicio programado para hoy, próximo servicio mañana, cliente nuevo en la cartera y seguimiento a un cliente VIP.
- **Qué hace.** Puede marcarlos como leídos o descartarlos uno a uno, y los puntos azules señalan los que aún no abrió.



Centro de avisos

Render con datos de demostración (Robolectric)

QUÉ SIGNIFICA

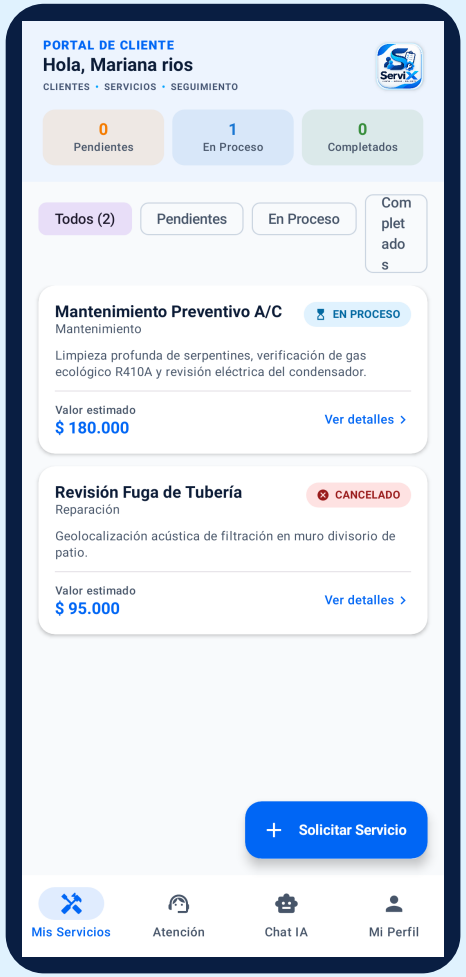
La app avisa en lugar de obligar al técnico a acordarse de todo.

Portal del cliente y bandeja de escalamientos

- **El portal.** El cliente ve contadores de sus servicios (pendientes, en proceso, completados), la lista con su valor estimado y un botón para solicitar un servicio.
- **El escalamiento.** Cuando el cliente pide atención humana, el caso aparece en la bandeja del administrador, con prioridad, motivo y el mensaje del cliente.
- **La respuesta.** El administrador pulsa Tomar caso y luego Responder: el caso pasa de Pendiente a En atención y, al terminar, a Resuelto.

QUÉ SIGNIFICA

El cliente se siente atendido y el técnico sabe qué caso debe responder primero.



Portal del cliente



Bandeja del administrador

Render con datos de demostración (Robolectric)

Cuatro capas: cada una hace una sola cosa

- **Presentación.** Las pantallas de Compose y el MainViewModel: solo muestran datos y reciben las acciones del usuario.
- **Dominio.** Los casos de uso y los modelos con las reglas del negocio. Define interfaces y no sabe nada de Firebase.
- **Datos.** Las implementaciones que hablan con Firebase, con Gemini (por Firebase AI Logic) y con Make por HTTP, más repositorios simulados para pruebas.
- **Núcleo.** El tema, la navegación y el AppContainer, que arma las dependencias a mano, sin Hilt.

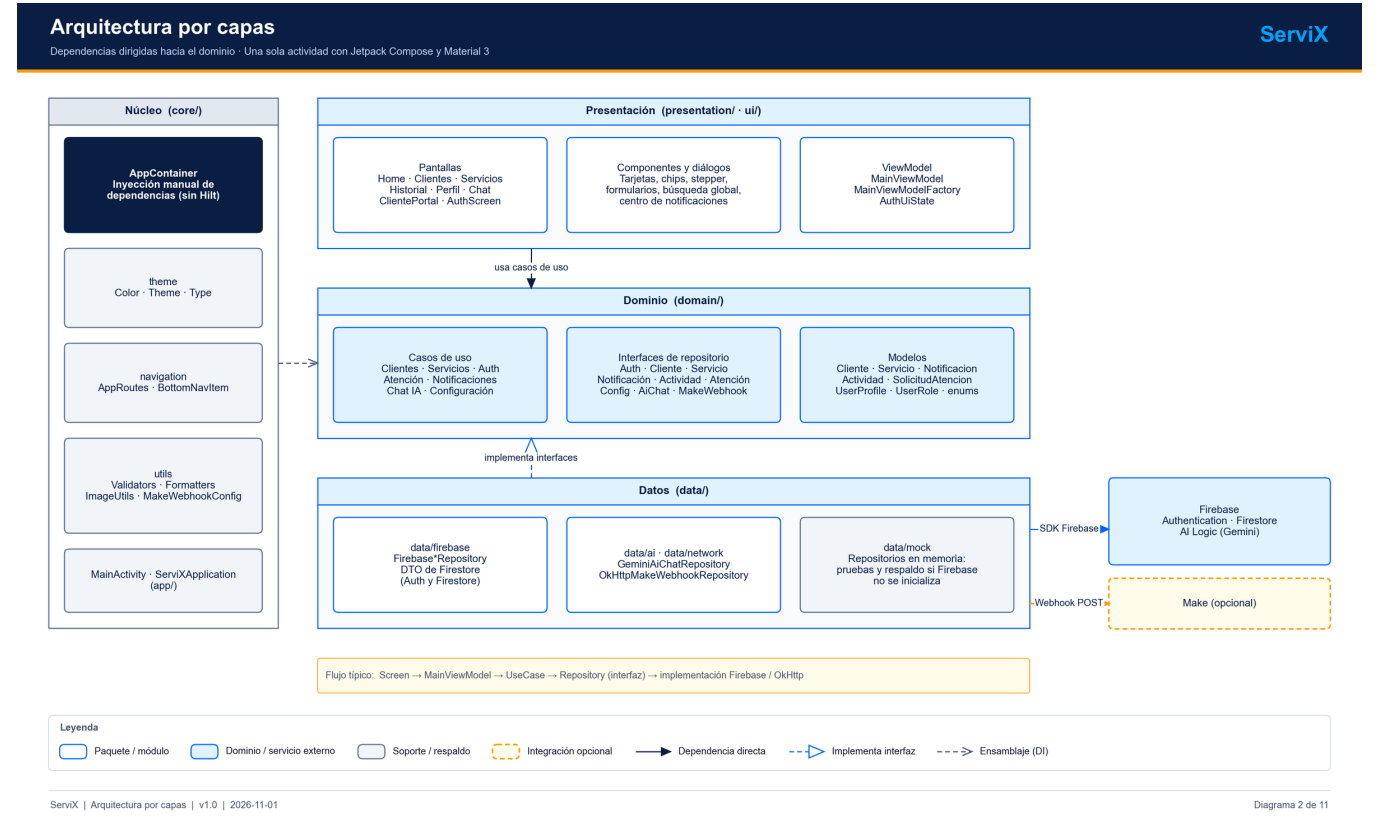


Diagrama de arquitectura por capas (diagrama 2 de la documentación)

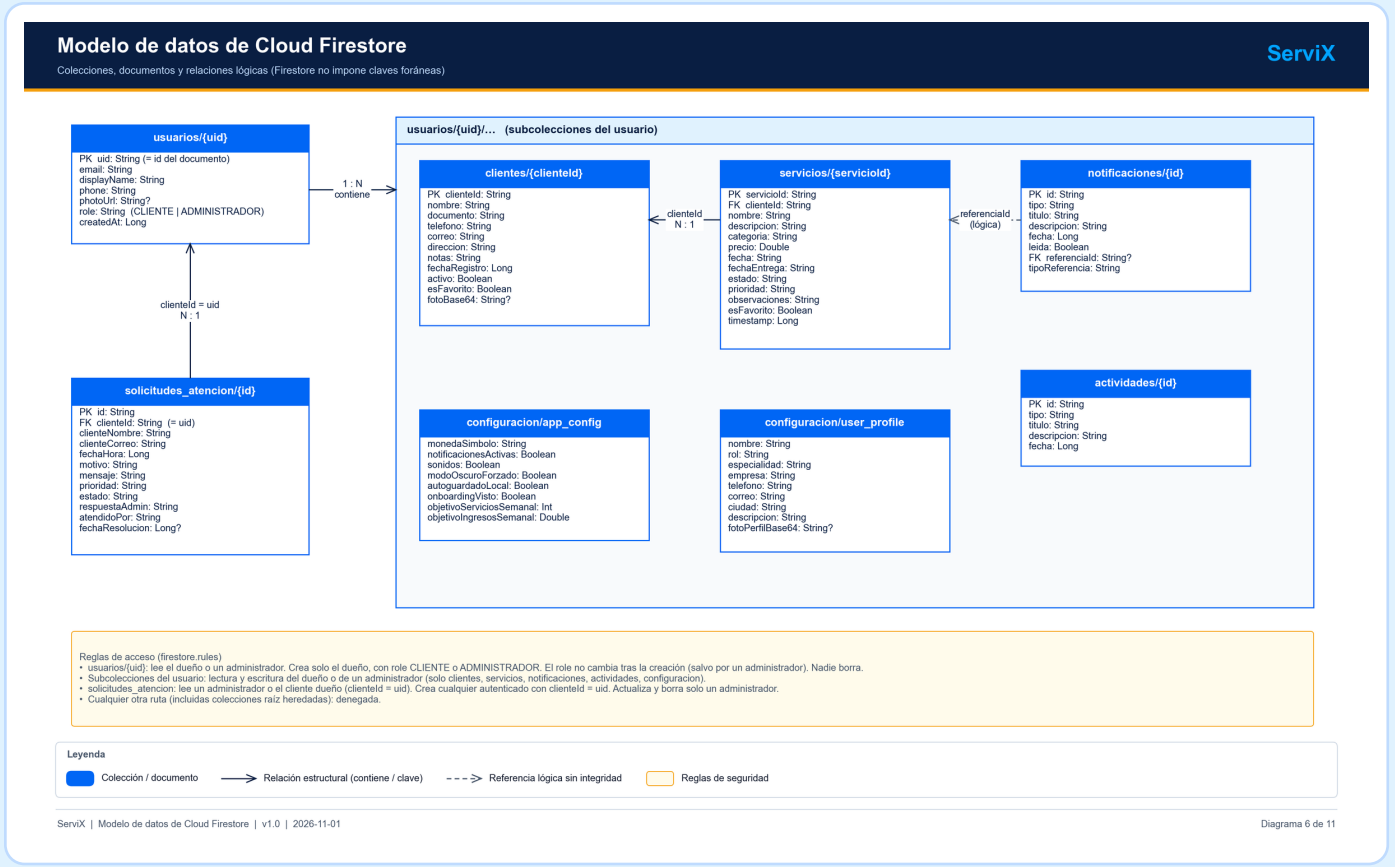
QUÉ SIGNIFICA

Se puede cambiar la base de datos sin tocar las pantallas, y probar la lógica sin Firebase.

DATOS Y REGLAS

Los datos viven en Firestore y las reglas los protegen

- **Qué se guarda.** Cada usuario tiene un documento con su rol y, dentro de él, sus clientes, servicios, notificaciones y actividades. Aparte hay una colección de solicitudes de atención.
- **Quién puede leer y escribir.** El dueño de los datos o un administrador. En las solicitudes de atención, el cliente solo ve las suyas y solo el administrador las modifica.
- **Todo lo demás.** Cualquier ruta que no esté en las reglas se deniega. Antes había reglas abiertas a cualquier persona; se reemplazaron y se desplegaron.



Modelo de datos de Cloud Firestore (diagrama 6 de la documentación)

QUÉ SIGNIFICA

El servidor de Firebase comprueba cada acceso: no depende solo de que la app se comporte bien.

Chat con IA: una decisión en cada mensaje

- **1. El mensaje.** La pantalla envía el texto al ViewModel y un caso de uso clasifica la intención por palabras clave.
- **2. ¿Queja o petición humana?** Si es queja, cancelación, problema, error de la app o pedido de asesor, se crea una solicitud de atención en Firestore con su prioridad y se avisa al usuario.
- **3. Si no, consulta a Gemini.** Se arma un contexto con los servicios, clientes y perfil de la persona y se envía a Gemini 2.5 Flash, con los últimos 6 mensajes.
- **4. Si algo falla.** La app muestra un error claro. Ya no se inventan respuestas: eso era un hallazgo de la auditoría.

QUÉ SIGNIFICA

La clave de IA ya no está dentro de la app y la IA nunca finge una respuesta.

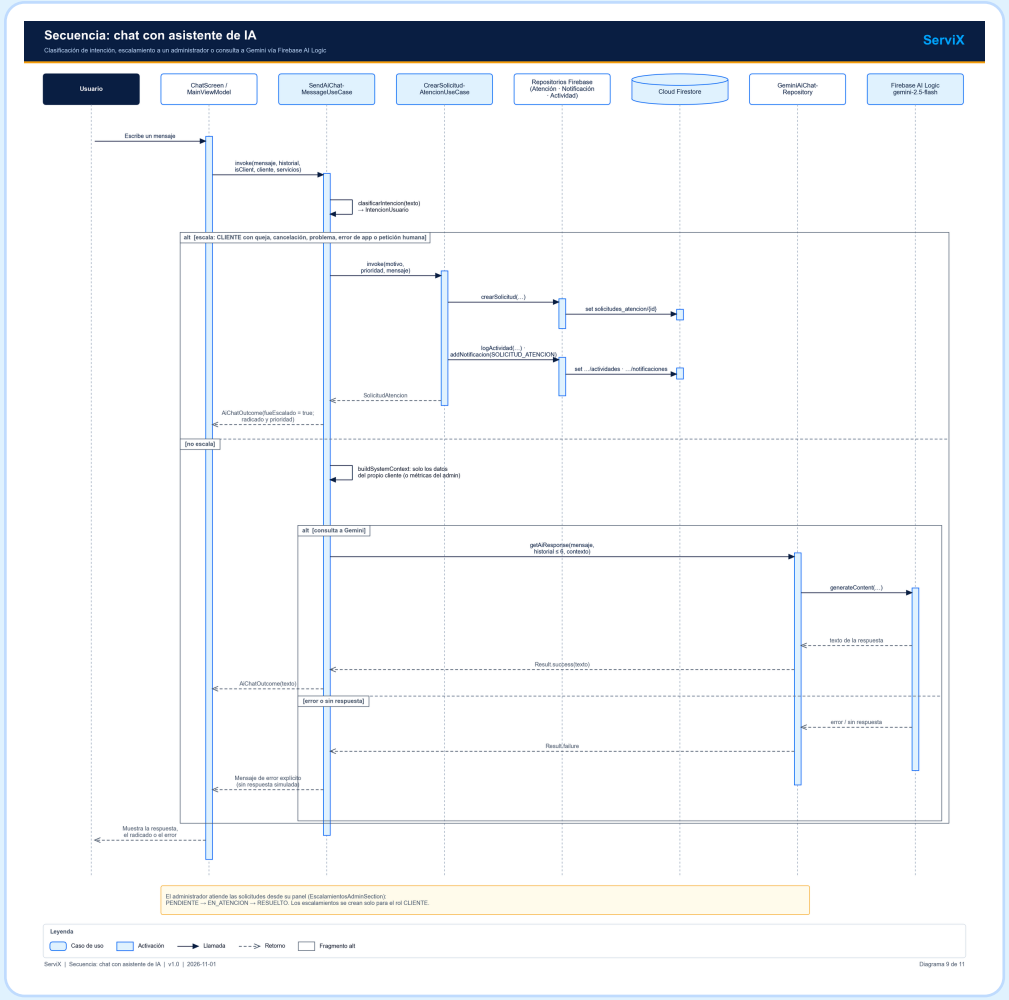


Diagrama de secuencia del chat con IA (diagrama 9 de la documentación)

SEGURIDAD

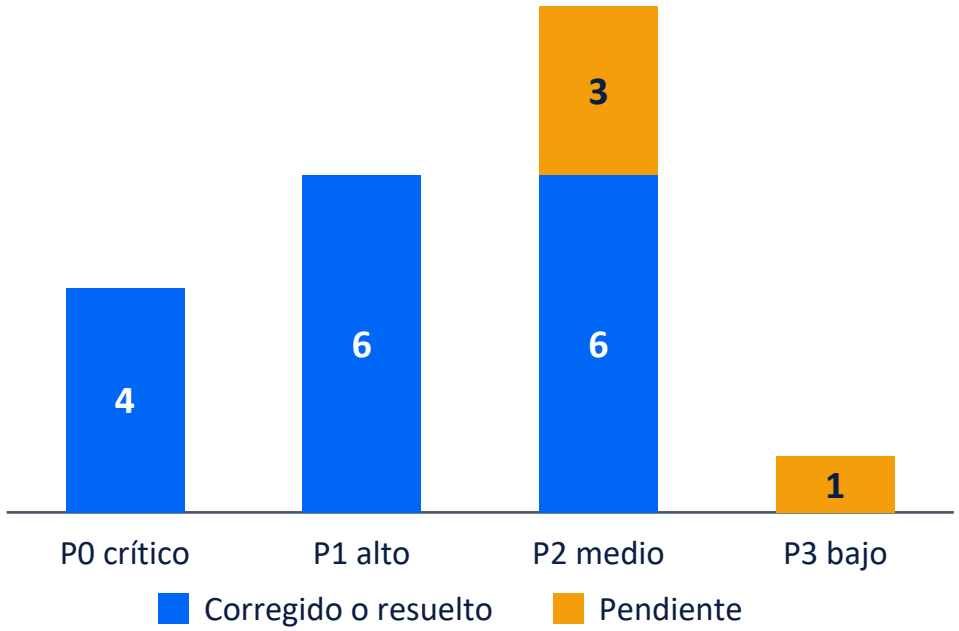
Auditamos el código de IA: 16 de 20 hallazgos corregidos

- **Qué encontramos.** Reglas de Firestore abiertas a cualquiera, una clave de Gemini dentro del APK, contraseñas de demostración y la URL de un webhook escritas en el código, y el rol de administrador asignado por defecto.
- **Qué hicimos.** Reglas por rol desplegadas, IA migrada a Firebase AI Logic, secretos fuera del código, rol por defecto Cliente y el rol tomado de la cuenta, no de la pantalla.
- **Qué queda.** Cuatro pendientes: el registro permite elegir administrador (H-15), AppContainer degrada en silencio a repositorios simulados (H-11), dos interfaces sin implementar (H-13) y una variable obsoleta en el archivo de ejemplo (H-16).

QUÉ SIGNIFICA

Todo lo crítico y lo importante (P0 y P1) está corregido; lo pendiente está documentado, no escondido.

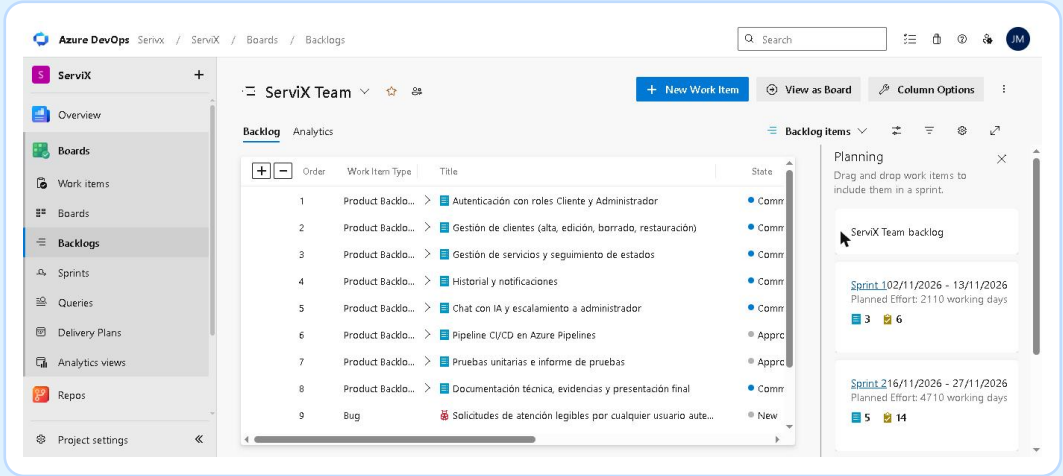
Estado de los 20 hallazgos por prioridad

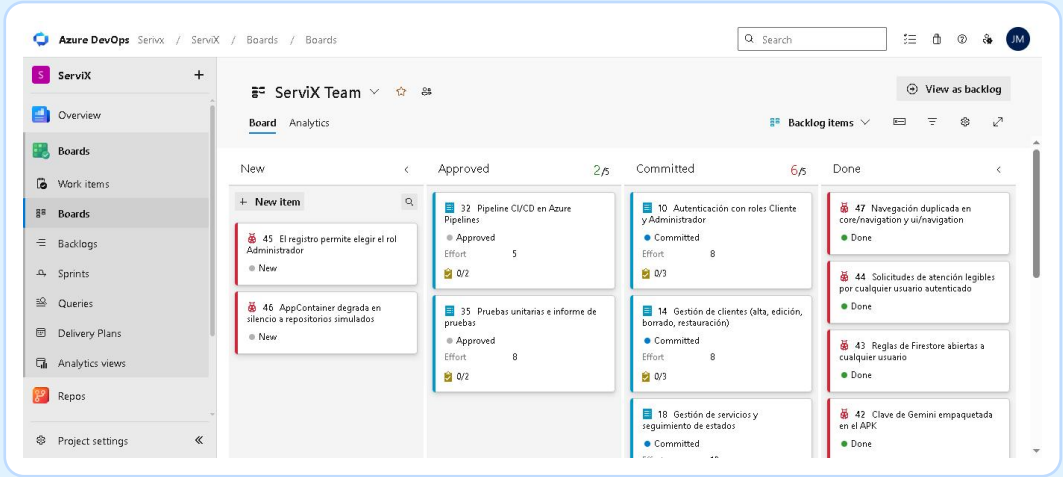


Fuente: informe de auditoría del proyecto (H-01 a H-20). Clasificación: P0 crítico, P1 alto, P2 medio, P3 bajo.

El trabajo del equipo, visible en un tablero

- **Equipo y roles.** Juan Esteban Olaya Muñoz es Developer, Nicolas Quiroz Gomez es Product Owner y Juan Sebastian Garzon Suarez es Scrum Master.
- **Product Backlog.** 12 PBIs (autenticación, clientes, servicios, historial, chat IA, pipeline, pruebas y documentación), 27 tareas y 6 bugs que nacieron de la auditoría.
- **Tablero.** Las columnas New, Approved, Committed y Done muestran el estado real: los bugs corregidos ya están en Done y los pendientes siguen en New.





Capturas reales de Azure DevOps (organización Serivx, proyecto ServiX)

QUÉ SIGNIFICA

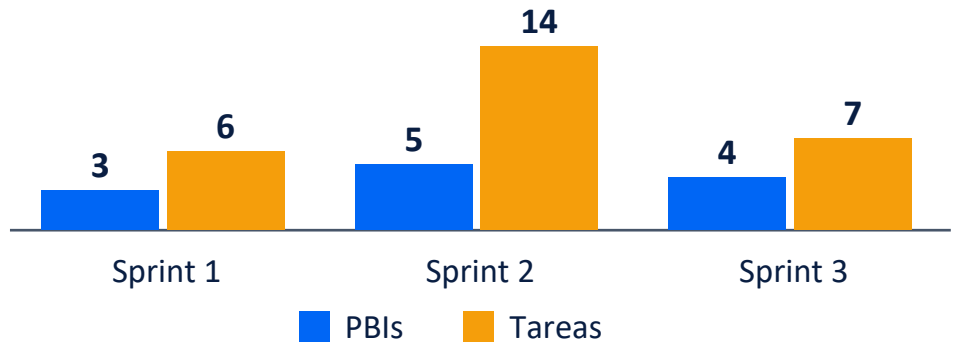
Cualquiera abre el tablero y entiende qué falta, qué se está haciendo y qué ya terminó.

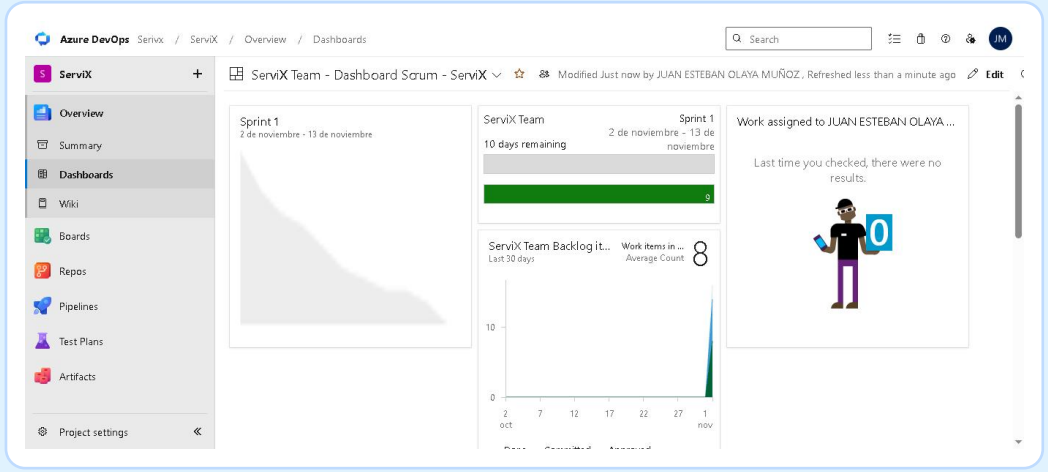
Tres sprints de dos semanas: diseño, desarrollo y entrega

- **Sprint 1: análisis y diseño.** Requisitos, arquitectura, modelo de datos y diagramas: 3 PBIs y 6 tareas.
- **Sprint 2: desarrollo.** Autenticación con roles, clientes, servicios, historial, avisos y chat: 5 PBIs y 14 tareas.
- **Sprint 3: integración y pruebas.** Pipeline, pruebas unitarias, informe y documentación: 4 PBIs y 7 tareas.

Las fechas de los sprints (02/11 al 11/12/2026) son provisionales hasta que el docente las confirme.

PBIs y tareas por sprint (12 PBIs y 27 tareas)





Dashboard Scrum real en Azure DevOps

QUÉ SIGNIFICA

Dividir el trabajo en tres entregas cortas permite revisar el avance y corregir el rumbo a tiempo.

Cada cambio se compila y se prueba solo, en la nube

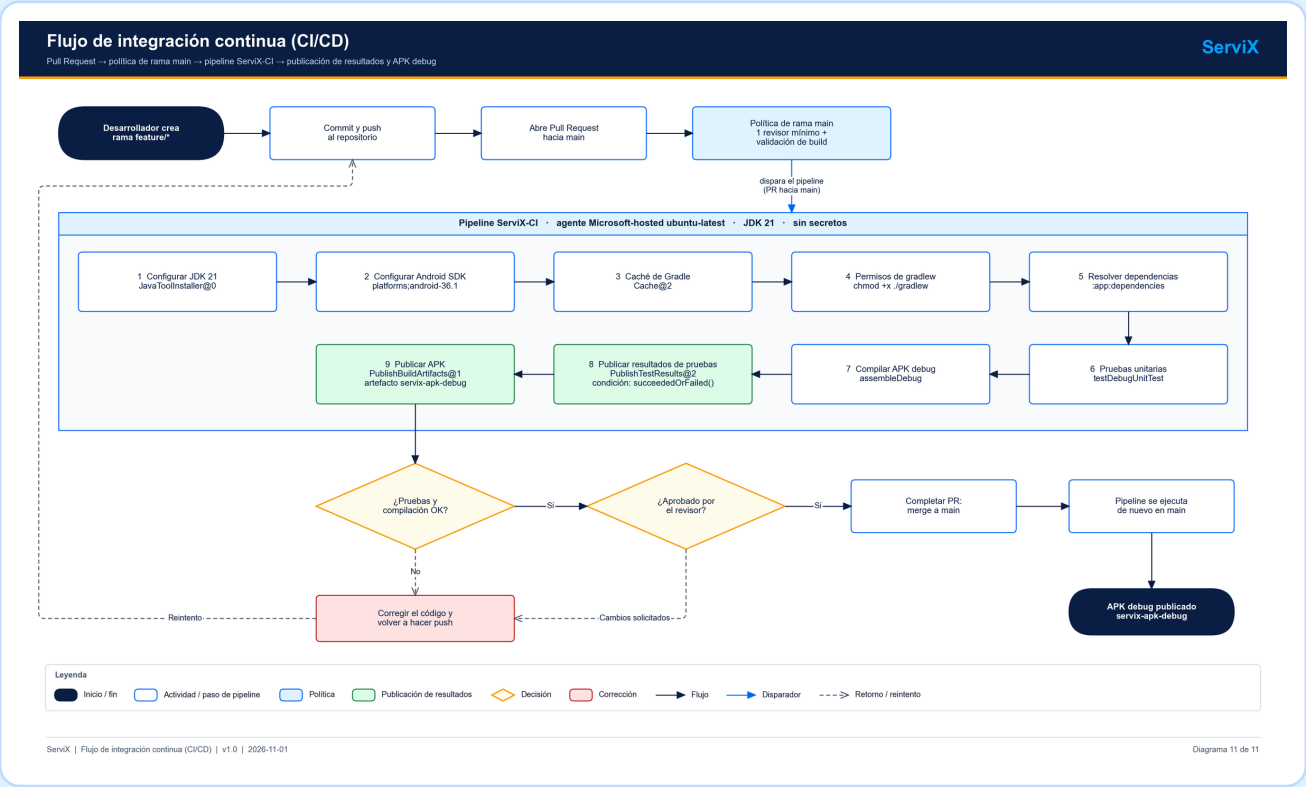


Diagrama del flujo CI/CD (diagrama 11 de la documentación)

QUÉ SIGNIFICA

Un pipeline que falla no es un fracaso: cada error nos enseñó algo concreto sobre el proyecto.

Ejecución 1: falló

El paso de dependencias se ejecutó sin indicar el módulo :app. Corrección: resolver las dependencias dentro de :app (PR 1).

Ejecución 2: falló

JDK 17 no sirve para el SDK 36 que exige Robolectric. Corrección: usar JDK 21 (PR 2).

Ejecución 3: exitosa

Pipeline 20261101.3: pruebas unitarias aprobadas al 100 % y APK de depuración publicado.

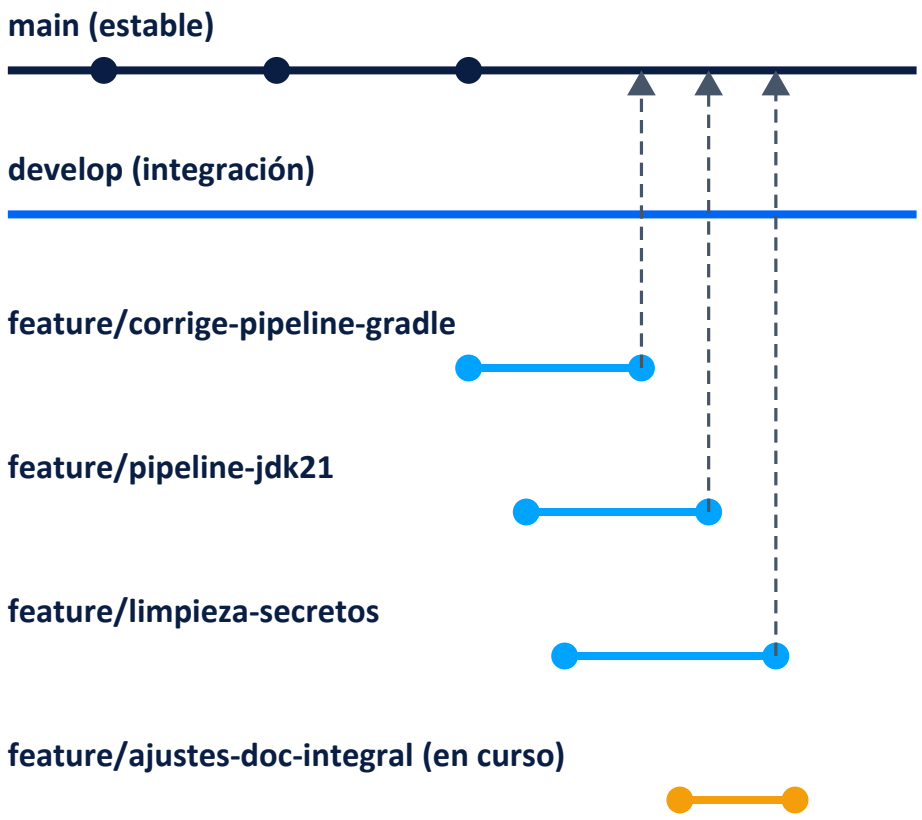
Git: ramas, Pull Requests y un historial limpio

- **Ramas.** main es la rama estable, develop sirve para integrar y cada cambio vive en una rama feature: corrige-pipeline-gradle, pipeline-jdk21, limpieza-secretos y ajustes-doc-integral.
- **Pull Requests.** Los dos arreglos del pipeline entraron por PR en Azure Repos, con la política de al menos un revisor.
- **Historial limpio.** Los commits usan prefijos como fix, security y chore. Los secretos que habían quedado en el historial se eliminaron reescribiéndolo. El código está en GitHub y en Azure Repos.

QUÉ SIGNIFICA

Cada cambio tiene autor, motivo y se puede revertir; y el historial ya no contiene secretos.

Ramas del repositorio (esquema)

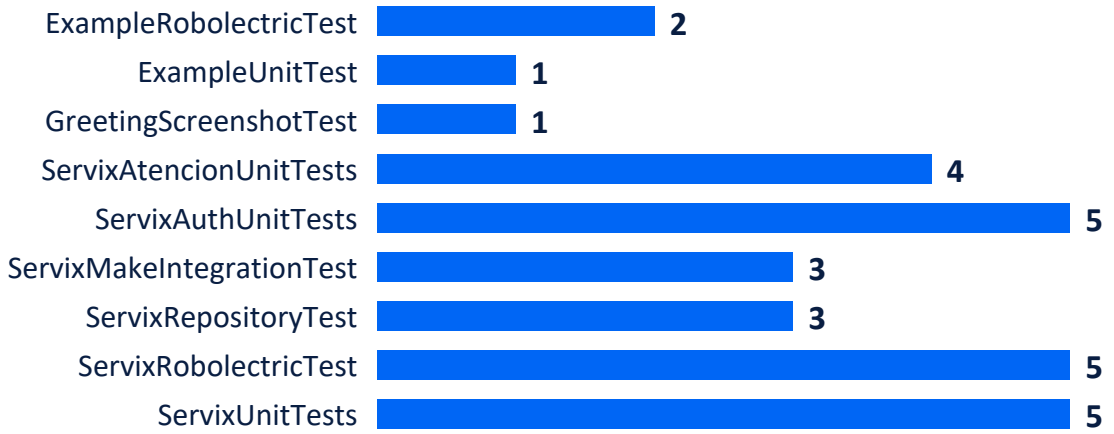


Flechas punteadas: fusión a main mediante Pull Request.

PRUEBAS

29 pruebas unitarias aprobadas; lo demás está pendiente

- **Qué se ejecutó.** 29 pruebas unitarias con Gradle el 31/10/2026 y, de nuevo, en Azure Pipelines (ejecución 20261101.3, 100 % aprobadas), sin fallos ni omitidas.
- **Qué cubren.** Casos de uso de clientes y servicios, autenticación, solicitudes de atención, repositorios, el envío al webhook con doble de prueba y la navegación entre pantallas con Robolectric.



QUÉ SIGNIFICA

Aprobado significa solo lo que se ejecutó: no afirmamos nada más.

Ejecutado

Sí Pruebas unitarias en Gradle (29 de 29)

Sí Pruebas unitarias en Azure (ejecución 20261101.3)

Sí Compilación de depuración

No ejecutado

No Pruebas en un dispositivo

No Android Lint

No Pruebas instrumentadas

No Chat contra Gemini real

No Inicio de sesión por rol en dispositivo

No Integración con Make real

RESULTADOS

Lo que entregamos, en cifras verificadas

29

pruebas unitarias aprobadas

16 de 20

hallazgos de auditoría corregidos

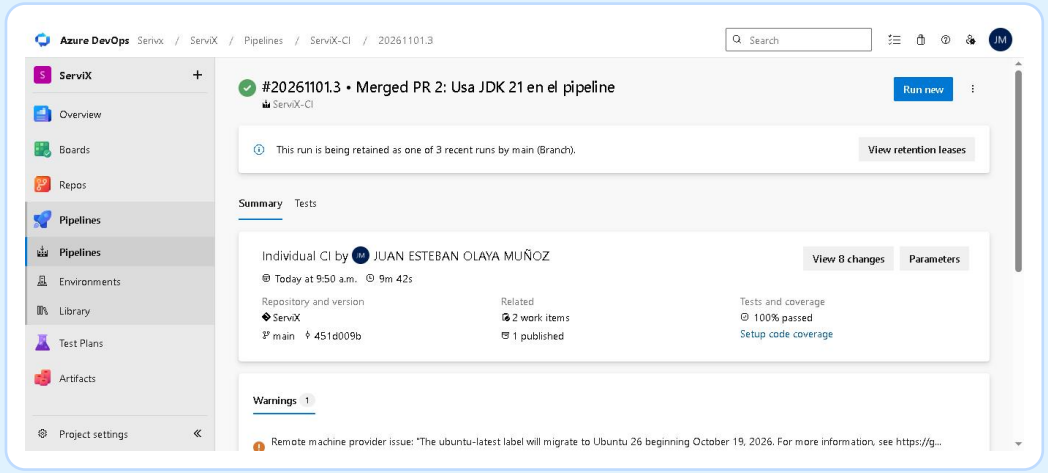
3

ejecuciones del pipeline: 2 fallos corregidos y 1 exitosa

12 · 27 · 6

PBIs, tareas y bugs en Azure

- **Producto.** Una app con dos roles, clientes, servicios con estados, historial, avisos, chat IA con escalamiento y portal del cliente, que compila en modo debug.
- **Proceso y calidad.** Scrum documentado en Azure DevOps, un pipeline que compila, prueba y publica el APK de depuración, y una auditoría que cerró todo lo crítico.



Ejecución 20261101.3 del pipeline (captura real de Azure DevOps)

QUÉ SIGNIFICA

Cada cifra de esta diapositiva se puede comprobar en un informe, una captura o un registro del proyecto.

DIFICULTADES Y SOLUCIONES

Tres problemas reales que cambiaron cómo trabajamos

4 GB de RAM y sin virtualización

Problema. El equipo no admite virtualización y tiene 4 GB de RAM: no hay emulador y Gradle llegó a quedarse sin memoria.

Qué hicimos. Pruebas de interfaz con Robolectric, que simula Android en la JVM, y capturas generadas con Roborazzi.

Resultado. 29 pruebas aprobadas y 16 renders de las pantallas. No sustituyen una prueba en un teléfono.

Secretos en el historial

Problema. La URL del webhook, un correo personal y contraseñas de demostración quedaron en el código y en commits antiguos.

Qué hicimos. Se sacaron del código (se leen de un archivo local o de una variable de entorno) y se reescribió el historial.

Resultado. El historial ya no los contiene. Quien tenga una copia anterior debe descartarla.

Clave de IA dentro del APK

Problema. La clave de Gemini estaba empaquetada en la app y viajaba en la URL: cualquiera podía extraerla.

Qué hicimos. Migración a Firebase AI Logic: la credencial la gestiona el proyecto Firebase, no la app.

Resultado. Ya no hay clave en el cliente. Queda pendiente activar App Check.

QUÉ SIGNIFICA

Cada dificultad terminó en una decisión técnica concreta, y todas dejaron lecciones que aplicamos en el resto del proyecto.

Lo que no hicimos y lo que viene después

Limitaciones honestas

- Las capturas son renders de Robolectric con datos de demostración, no de un dispositivo.
- No se ejecutaron pruebas en dispositivo, Lint, pruebas instrumentadas ni el chat con Gemini real.
- El registro todavía permite elegir el rol de administrador (H-15).
- Make no se probó con Make real, y una URL dentro de un APK se puede extraer.
- Las fechas de los sprints son provisionales.

Trabajo futuro

- Asignar roles con claims de administrador desde un backend.
- Probar en un teléfono físico, ejecutar Lint y las pruebas instrumentadas.
- Activar App Check y restringir la clave de API de Firebase.
- Evitar que la app degrade en silencio a repositorios simulados (H-11).
- Mover Make detrás de un backend y decidir qué hacer con Retell y n8n (H-13).

QUÉ SIGNIFICA

Tener claro lo pendiente es parte del trabajo: así el siguiente equipo sabe por dónde empezar.

CONCLUSIONES

Construir con IA funciona, pero hay que auditar

1

El producto.

ServiX ordena clientes, servicios, cobros y atención en una app con dos roles y un asistente con IA que sabe cuándo pasar el caso a una persona.

2

El proceso.

Scrum, Git y CI/CD hicieron el trabajo visible, repetible y verificable: cada cambio se compila y se prueba solo.

3

La lección.

El código generado con IA puede traer reglas abiertas, claves y contraseñas expuestas. Una auditoría cuidadosa los encontró y los corrigió; lo pendiente está documentado.

QUÉ SIGNIFICA

La IA acelera el desarrollo, pero la responsabilidad por la calidad y la seguridad sigue siendo del equipo.

Gracias

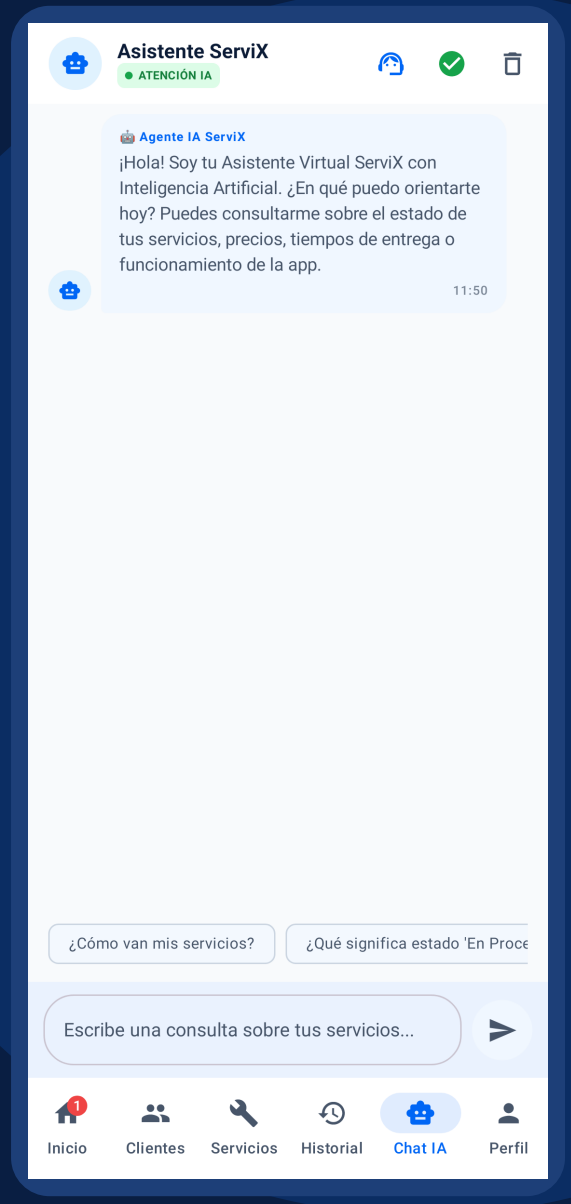
¿Preguntas?

Juan Esteban Olaya Muñoz · Developer

Nicolas Quiroz Gomez · Product Owner

Juan Sebastian Garzon Suarez · Scrum Master

Universidad de San Buenaventura · Optativa II
Equipo ServiX



Render con datos de demostración (Robolectric)